

Scheduling: The Switching Mechanism

2026 Semester 1 SOFTENG 370: Operating Systems
Talia Xu

Lecture 8
2.0.0

Based on original slides by Professor Jon Eyolfson.

This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/)

How does the OS run a process?

1. OS creates PCB
2. OS allocates memory
3. OS initializes registers
4. Process is in ready state <- (OS kernel), runs.
5. When the process runs, the process executes directly on CPU (process runs), user

There is a transfer of CPU ownership

How Does the OS Switch Between Processes?

A process has the ownership of the CPU when it runs.

To switch to a different process, the OS needs to regain control.

A simple function call

→ top of stack ↑.

A new stack frame is pushed to the stack and the SP updated.

The old PC pushed to the stack and updated.

How about a system call?

The OS provides user programs with controlled access to system functionality.

`write()` -

`fork()` -

`pipe()` -

How is a system call different from a function call?

How about a system call?

CPU hardware has multiple privilege levels

- One to run user code: user mode
- One to run OS code like system calls: kernel mode
- Some instructions execute only in kernel mode

Kernel does not trust user stack

- Uses a separate kernel stack when in kernel mode

Kernel does not trust user provided addresses to jump to

- How does the kernel know which code to run?

System Call Mechanism: Trap

A special trap instruction handles system calls
(usually hidden from user by libc)

Trap instruction

- Move CPU from user mode to kernel mode
(process still owns CPU, OS controls CPU on behalf of process) *(transition of cpu ownership).*
- Switch to kernel stack
- Save context on kernel stack
- Jump to trap handler function in OS code *(interrupt handler).*

System Call Mechanism: Trap

Trap instruction is executed:

- System call
- Program fault (i.e., segmentation fault, page fault)
- Interrupt (hardware).

How does the kernel know which code to run?

Trap Mechanism

OS at boot .

initialize trap table

Hardware

Remember address of
syscall handlers

OS at run

...

Hardware

kernel .

- save regs
(to kernel stack)
- move to kernel mode (CPU) .
- jump to trap handler

Handle trap
Do work of syscall
return from trap

...

pre-run .

user program .

Program

Run main()

...
invoke system call
trap into OS

...

Return from trap

When OS is done handling syscall or interrupt, it calls a special instruction return-from-trap

- Restore context of CPU registers from kernel stack
- Change CPU privilege from kernel mode to user mode
- Restore PC and jump to user code after trap

User process unaware that it was suspended, resumes execution as always

⇒ Before returning to user mode, OS checks if it must switch to another process

Switching Process: Cooperative

⇒ Program voluntarily transfers control of CPU to OS.

OS can then decide whether to transfer control to another process.

Examples:

- system call (if blocking)
- program fault

The OS cannot rely on user programs to always give up control voluntarily.

- What if the process gets stuck in an infinite loop?

while (1);

Switching Process: Non-Cooperative (support of hardware).

⇒ A timer interrupt device is started at boot time.

The timer goes off periodically. (interrupt).

The interrupt handler first services the timer, then checks if the current process has run for too long.

OS to have
ownership.

The OS Scheduler

OS scheduler has two parts

- • Policy to pick which process to run
- Mechanism to switch to that process

Non-preemptive (cooperative) schedulers switch only if a process is blocked or terminated.

Preemptive (non-cooperative) schedulers can switch even when process is ready to continue.

Mechanism of context switch

Example: The OS switches from Process A to Process B

1. Save context of A on kernel stack
2. Switch SP to kernel stack of B
3. Restore context from kernel stack of B
4. return-from-trap

Scheduling Policy

There are many ready processes, which one to run next?

→ CPU burst: The continuous CPU time requested by a process

If a process comes back after I/O wait, it counts as a fresh CPU burst

We assume no I/O for now.

Metrics

- ✓ Minimize waiting time and response time
Don't have a process waiting too long (or too long to start)
- Maximize CPU utilization
Don't have the CPU idle
- Maximize throughput
Complete as many processes as possible
- Fairness
Try to give each process the same percentage of the CPU

First Come First Served (FCFS)

The most basic form of scheduling

The first process that arrives gets the CPU

Processes are stored in a FIFO queue in arrival order

A Gantt Chart Illustrates the Schedule

Consider the following processes:

Process	Arrival Time	Burst Time
P ₁	0	<u>7</u>
P ₂	0	4
P ₃	0	1
P ₄	0	4

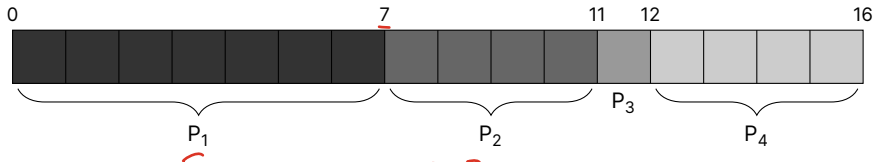
$$P_1 = 0$$

$P_2: 7$

P3: 11

P4: 12.

Assume, $P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4$. For FCFS, our schedule is:



What is the average waiting time?

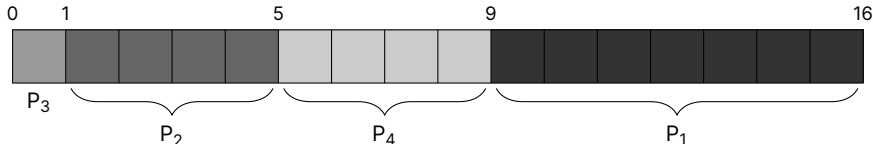
$$\frac{0+7+11+12}{4} = \cancel{7.5} \quad 7.5.$$

What Happens to Our Waiting Time with a Different Arrival Order

Consider the same processes:

Process	Arrival Time	Burst Time
P ₁	0	7
P ₂	0	4
P ₃	0	1
P ₄	0	4

Assume, $P_3 \rightarrow P_2 \rightarrow P_4 \rightarrow P_1$. For FCFS, our schedule is:



What is the average waiting time now?

3.75

Shortest Job First (SJF)

A slight tweak to FCFS, we always schedule the job with the shortest burst time first

We're still assuming no preemption

FCFS = tie-breaker.

SJF Minimizes the Average Wait Time over FCFS

Consider the same processes with different arrival times:

Process	Arrival Time	Burst Time
→ P ₁	0	7
P ₂	2	4
P ₃	4	<u>1</u>
P ₄	<u>5</u>	4

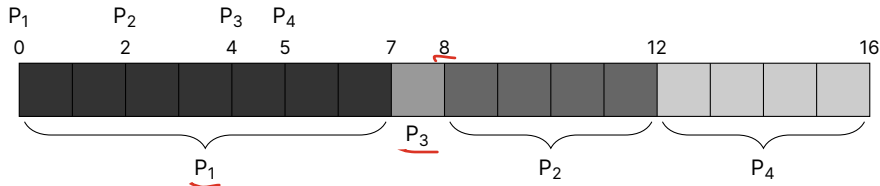
P₁ = 0.

P₂ = 6.

P₃ = 3.

P₄ = 7.

For SJF, our schedule is (arrival on top):



Average waiting time: $\frac{0+6+3+7}{4} = 4$

SJF is Not Practical

It is provably optimal at minimizing average wait time (if no preemption)

You will not know the burst times of each process

You could use the past to predict future executions

You may starve long jobs (they may never execute) (low fairness).

Shortest Remaining Time First (SRTF) (pre-emptive).

Changing SJF to run with preemptions requires another tweak

We'll assume that our minimum execution time is one unit.

Similar to SJF, this optimizes the average waiting time

SRTF Reduces the Average Wait Time Compared to SJF

Consider the same processes and arrival times as SJF:

Process	Arrival Time	Burst Time
P ₁	0	7
P ₂	2	4
P ₃	4	1
P ₄	5	4

(FCFS tie-breaker)

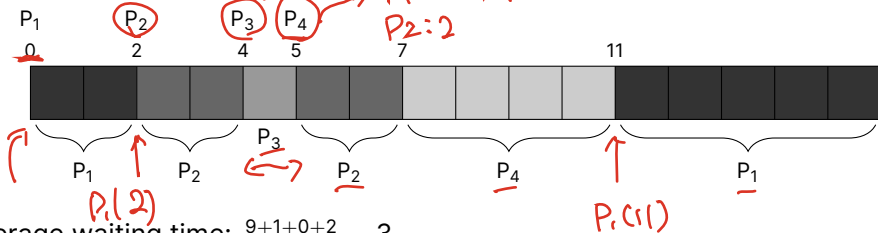
$$P_1: 0 + 7 = 7.$$

$$P_2: 0 + 1 = 1.$$

$$P_3: 0.$$

$$P_4: 2.$$

For SRTF, our schedule is (arrival on top):



Round-Robin (RR)

So far we haven't handled fairness (it's a trade off with other metrics)

The operating system divides execution into time slices (or quanta)

An individual time slice is called a quantum

⇒ Maintain a FIFO queue of processes similar to FCFS

Preempt if still running at end of quantum and re-add to queue

⇒ What are practical considerations for determining quantum length?

time slice. RR with a Quantum Length of 3 Units

context switch: 7

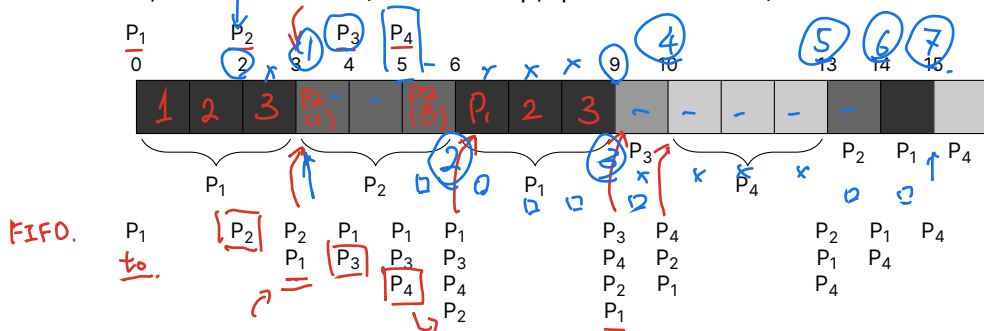
Avg waiting time: $\frac{8+8+5+7}{4} = \sim$

Avg response time.

$\frac{0+1+5+5}{4} = \sim$

Process	Arrival Time	Burst Time
P ₁	0	7
P ₂	2	4
P ₃	4	1
P ₄	5	4

For RR, our schedule is (arrival on top, queue on bottom):



Metrics for RR (3 Unit Quantum Length)

Number of context switches: 7

Average waiting time: $\frac{8+8+5+7}{4} = 7$

Average response time: $\frac{0+1+5+5}{4} = 2.75$

Note: on ties (a new process arrives while one is preempted), favor the new one

RR with a Quantum Length of 1 Units

of context switch: 14.

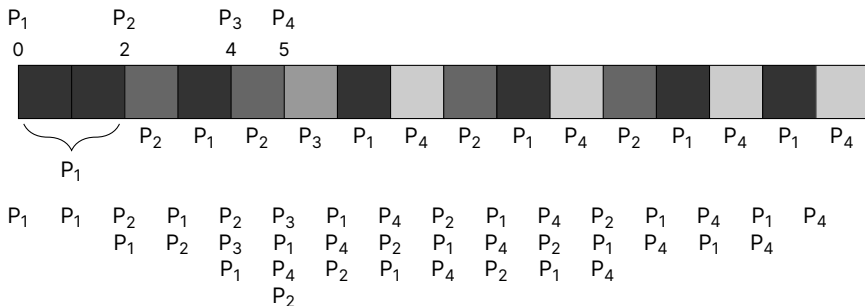
avg. waiting time $\frac{8+6+1+0}{4}$.

avg. response time.

$\frac{0+0+1+2}{4}$.

Process	Arrival Time	Burst Time
P ₁	0	7
P ₂	2	4
P ₃	4	1
P ₄	5	4

For RR, our schedule is (arrival on top, queue on bottom):



Metrics for RR (1 Unit Quantum Length)

Number of context switches: 14

Average waiting time: $\frac{8+6+1+7}{4} = 5.5$

Average response time: $\frac{0+0+1+2}{4} = 0.75$

RR with a Quantum Length of 10 Units

response time:

worse if Q higher.

if QL too small.

context switch overhead is high.

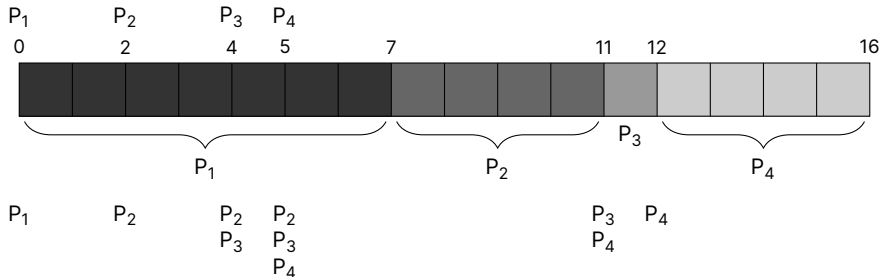
Process	Arrival Time	Burst Time
P ₁	0	7
P ₂	2	4
P ₃	4	1
P ₄	5	4

context switch: 3.

waiting time: $\frac{0+5+7+7}{4}$

response time: $\frac{0+5+7+7}{4}$

For RR, our schedule is (arrival on top, queue on bottom):



Metrics for RR (10 Unit Quantum Length)

Number of context switches: 3

Average waiting time: $\frac{0+5+7+7}{4} = 4.75$

Average response time: $\frac{0+5+7+7}{4} = 4.75$

Note: in this case it's the same as FCFS without preemptions

RR Performance Depends on Quantum Length and Job Length

RR has low response good interactivity

=> Fair allocation of CPU

Low average waiting time (when job lengths vary)

The performance depends on the quantum length

Too high and it becomes FCFS

Too low and there's too many context switches (overhead)

<= RR has poor average waiting time when jobs have similar lengths

Scheduling Involves Trade-Offs

We looked at few different algorithms:

- First Come First Served (FCFS) is the most basic scheduling algorithm
- Shortest Job First (SJF) is a tweak that reduces waiting time
- Shortest Remaining Time First (SRTF) uses SJF ideas with preemptions
- SRTF optimizes lowest waiting time (or turnaround time)
- Round-robin (RR) optimizes fairness and response time

waiting
time